

Lab 4

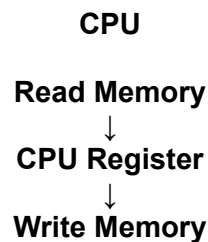
Week 4

DMA Engine Design

RTL design of a simple AXI-based DMA engine (memory-to-memory transfer).

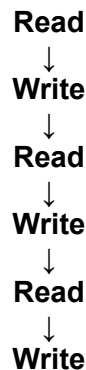
Direct Memory Access - DMA Engine

CPU Method:



Every single word passes through the CPU.

For 1000 words:



Is the CPU doing any computation?

The CPU is acting like a delivery person carrying boxes from one room to another.

That is a waste of CPU cycles.

Here comes the DMA:

Instead of the CPU copying data, the CPU simply says:

Copy

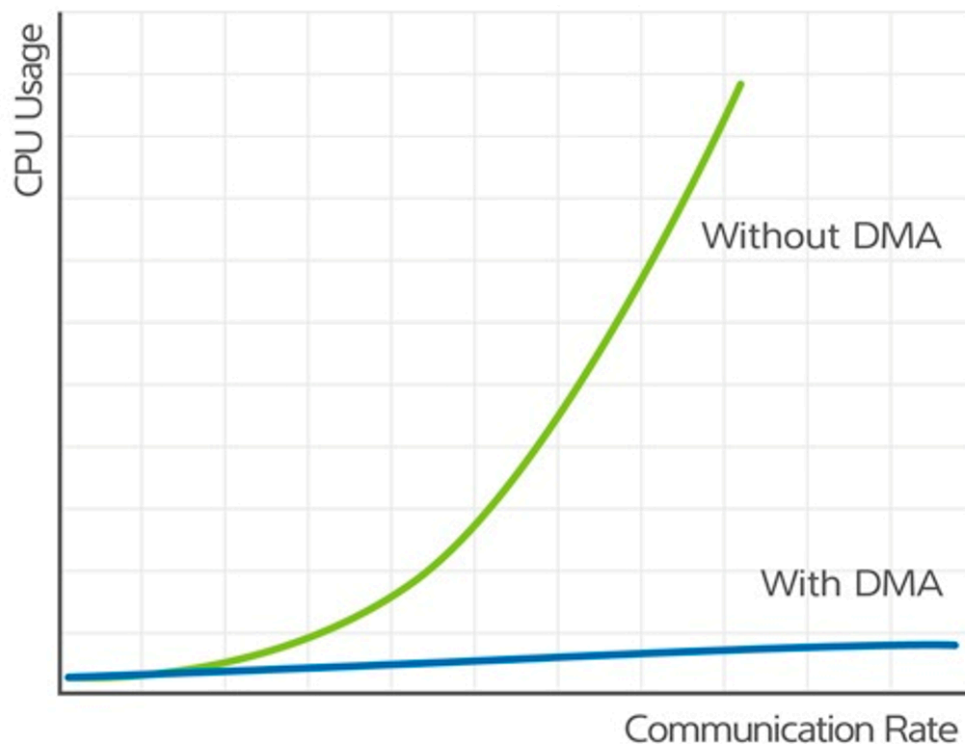
Source = 0x1000

Destination = 0x2000

Length = 1000

DMA then performs the transfer.

The CPU can continue executing other programs.



Direct Memory Access (DMA) is a process of transferring data from one memory location to another without the direct involvement of the processor (CPU). The main benefit of using DMA is more efficient data movement in the embedded system.

Principle of Operation

There are many different types of DMA implementations, some for particular use cases. In this Lab, we will focus on the general principles of operation. Let's start with the simple system shown below in Fig 1.

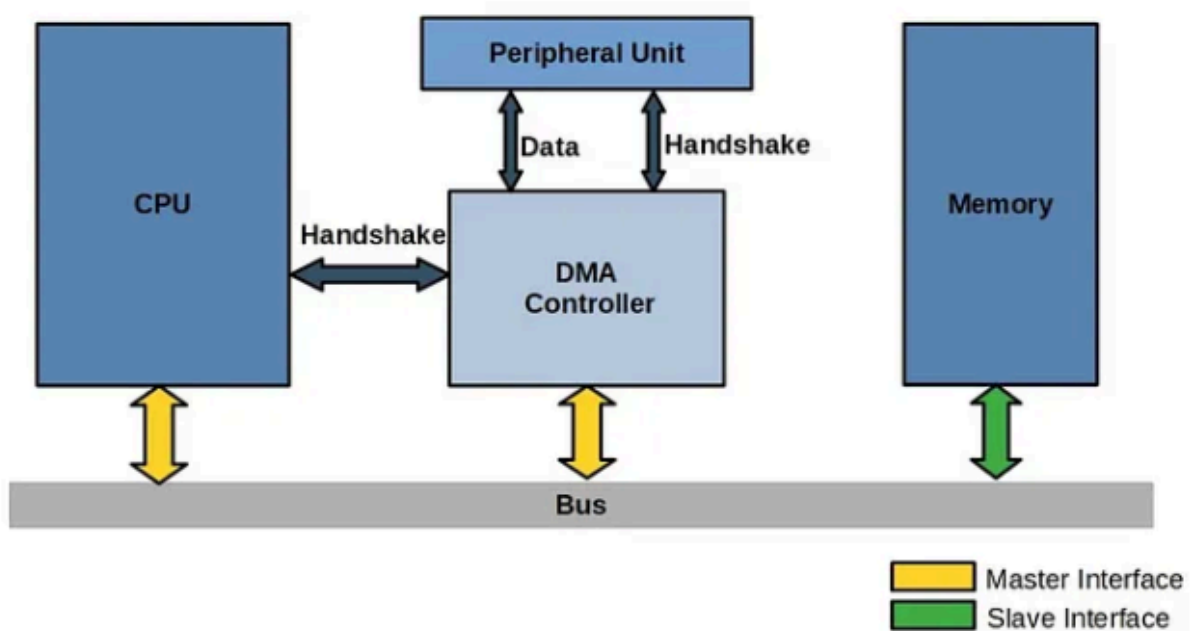


Fig 1. Simplified DMA block diagram

A DMA controller is the functional unit that performs the operations for directly accessing the memory. On the simplified block diagram (Fig 1), we have a CPU, RAM, a peripheral unit, and a DMA controller. All except the peripheral unit are connected to the same bus. As the CPU and the DMA controller must be able to initiate transfers, they have master interfaces. Although the goal is to have a DMA that operates independently, the CPU has to configure the DMA controller to perform transfers. The DMA controller can be dedicated to a specific DMA-capable peripheral unit (Fig 1) or a more general DMA able to access various memory-mapped peripherals.

Example: Saving ADC results in RAM using a DMA

Let's have an ADC peripheral unit that produces a new conversion result every 4 microseconds. The CPU has configured a DMA controller to read this new data and store it in RAM memory. The source address for the transfers is always the ADC data register. The destination address is an address in the RAM memory. The DMA controller increases this destination address with each consecutive transfer. The ADC indicates to the DMA that new data is available using a dedicated interface. When the DMA detects that new data is available, it proceeds by reading the data register of the ADC. The read data is then stored in the DMA FIFO buffer. The DMA controller checks that the memory bus is not busy and then transfers the data from the FIFO buffer to the destination address in the RAM. This concludes a single DMA transfer.

Type of Transfers

The most common configuration in modern microcontrollers uses a single memory address space, where all types of memories (Code Flash, RAM, Data Flash, etc.) and peripheral units are mapped into the same address space. For the DMA controller, all transfers are memory-to-memory. For us as developers, however, by knowing what functionality is behind each memory address, we can use the DMA for the following types of transfers:

- memory to peripheral unit and vice versa
- memory to memory
- peripheral unit to peripheral unit

Each DMA controller has FIFO buffers. They are used in case a busy resource prevents the transfer from completion. Using FIFO allows the DMA controller to perform two-step transfers:

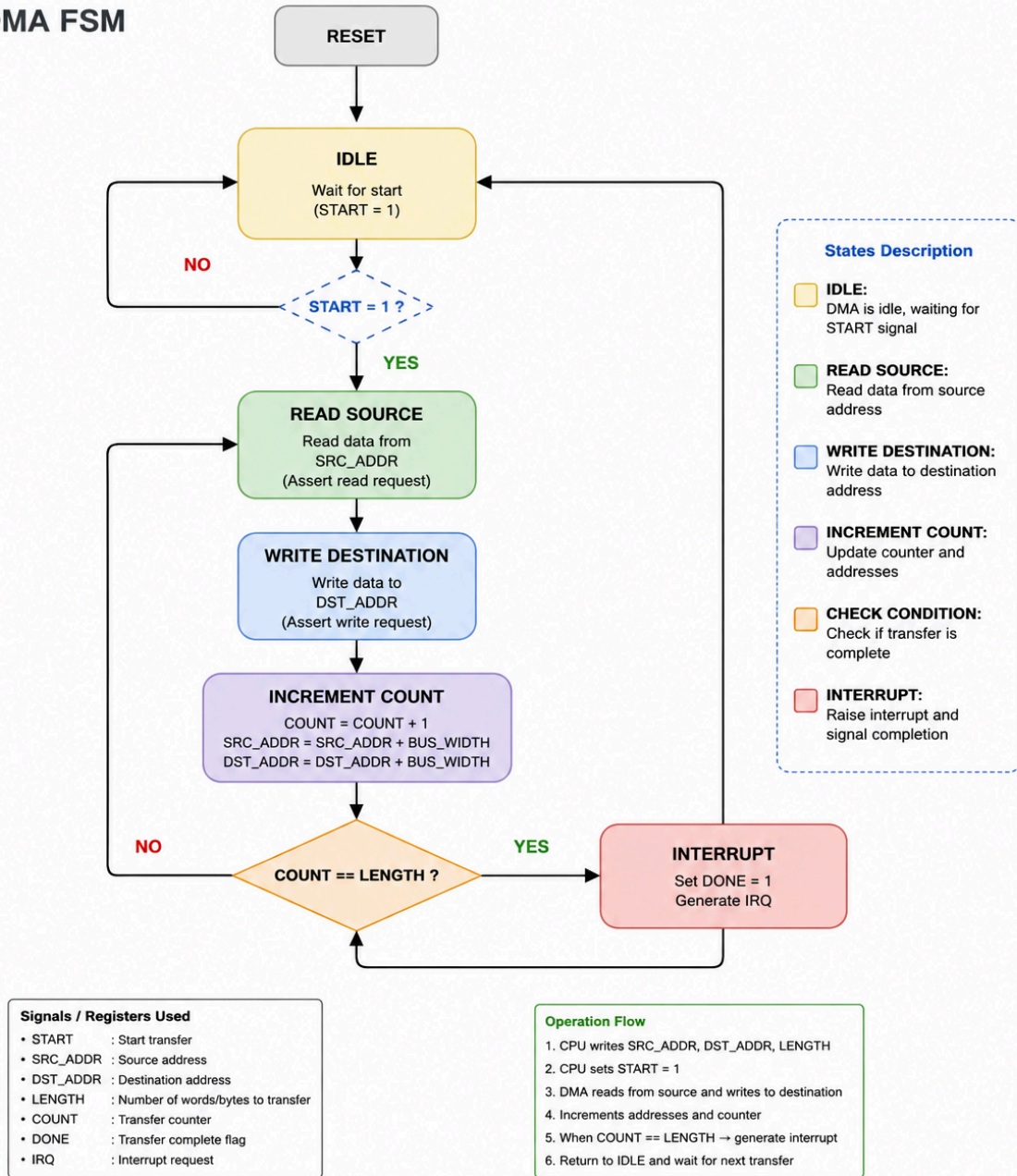
- Read access — data is transferred from the source address to the DMA FIFO
- Write access — data is transferred from the DMA FIFO to the destination address



Why Use DMA?

DMA moves data from one memory address to another. When used correctly, it can improve the efficiency of an embedded system. The CPU can be more focused on performing calculations without having to waste too many instruction cycles transferring data, which can improve the speed of our program.

DMA FSM



State Transition Table (Next State Logic)

Current State	Condition	Next State
STATE_IDLE	start_transfer && (length_init > 0)	STATE_READ
STATE_READ	No condition (always)	STATE_WAIT_READ
STATE_WAIT_READ	bus_op_done==1	STATE_WRITE
STATE_WRITE	No condition (always)	STATE_WAIT_WRITE
STATE_WAIT_WRITE	bus_op_done	STATE_INC_ADDR
STATE_WAIT_WRITE	!bus_op_done	STATE_WAIT_WRITE
STATE_INC_ADDR	transfer_count == 0	STATE_DONE
STATE_INC_ADDR	transfer_count != 0	STATE_READ
STATE_DONE	No condition (always)	STATE_IDLE
Default	No condition	STATE_IDLE

Output Control Signals Table

Current State	bus_read_req	bus_write_req	transfer_done	transfer_active	Address/Count Update
STATE_IDLE	0	0	0	0	Load initial values on start
STATE_READ	1	0	0	1	No update
STATE_WAIT_READ	0	0	0	1	No update
STATE_WRITE	0	1	0	1	No update
STATE_WAIT_WRITE	0	0	0	1	No update

STATE_INC_ADDR	0	0	0	1	src_addr += 4, dst_addr += 4, transfer_count -= 1
STATE_DONE	0	0	1	0	No update

Register Update Summary

Register	Updated In	Update Condition	Value
current_src_addr	STATE_IDLE	start_transfer	src_addr_init
current_src_addr	STATE_INC_ADDR	Always	current_src_addr + 4
current_dst_addr	STATE_IDLE	start_transfer	dst_addr_init
current_dst_addr	STATE_INC_ADDR	Always	current_dst_addr + 4
transfer_count	STATE_IDLE	start_transfer	length_init
transfer_count	STATE_INC_ADDR	Always	transfer_count - 1

```

module dma_fsm (
    input wire clk,
    input wire rst,

    input wire start_transfer,
    input wire [31:0] src_addr_init,
    input wire [31:0] dst_addr_init,
    input wire [31:0] length_init,

    input wire bus_op_done,
    output reg [31:0] current_src_addr,
    output reg [31:0] current_dst_addr,
    output reg bus_read_req,
    output reg bus_write_req,

    output reg transfer_done,

```



```

output reg transfer_active,

input wire [31:0] read_data_buffer
);

localparam STATE_IDLE    = 3'b000;
localparam STATE_READ    = 3'b001;
localparam STATE_WAIT_READ = 3'b010;
localparam STATE_WRITE   = 3'b011;
localparam STATE_WAIT_WRITE = 3'b100;
localparam STATE_INC_ADDR = 3'b101;
localparam STATE_DONE    = 3'b110;

reg [2:0] state;
reg [2:0] next_state;
reg [31:0] transfer_count;

always @(posedge clk or negedge rst) begin
    if (!rst)
        state <= STATE_IDLE;
    else
        state <= next_state;
end

always @(*) begin
    next_state = state;
    case (state)
        STATE_IDLE: begin
            if (start_transfer && (length_init > 0))
                next_state = STATE_READ;
        end

        STATE_READ:    next_state = STATE_WAIT_READ;
        STATE_WAIT_READ: if (bus_op_done) next_state = STATE_WRITE;
        STATE_WRITE:   next_state = STATE_WAIT_WRITE;
        STATE_WAIT_WRITE: if (bus_op_done) next_state = STATE_INC_ADDR;

        STATE_INC_ADDR: begin
            if (transfer_count == 0)
                next_state = STATE_DONE;
            else
                next_state = STATE_READ;
        end

        STATE_DONE: next_state = STATE_IDLE;
        default:    next_state = STATE_IDLE;
    endcase
end

// ork on 3 signal. current_src_addr , current_dst_addr and transfer_count.
always @(posedge clk or negedge rst) begin
    if (!rst) begin

```

```

        //to do
    end else begin
        case (state)
            STATE_IDLE: begin
                if (start_transfer) begin
                    // to do. Assign value
                end
            end

            STATE_INC_ADDR: begin
                //to do, current_src_addr , current_dst_addr and transfer_count.
            end
        endcase
    end
end

always @(posedge clk or negedge rst) begin
    if (!rst) begin
        bus_read_req    <= 1'b0;
        bus_write_req   <= 1'b0;
        transfer_done    <= 1'b0;
        transfer_active  <= 1'b0;
    end else begin
        bus_read_req <= 1'b0;
        bus_write_req <= 1'b0;
        transfer_done <= 1'b0;

        case (state)
            STATE_IDLE:
                // to do , transfer_active

            STATE_READ: begin
                // to do. bus_read_req , transfer_active.
            end

            STATE_WAIT_READ:
                //to do. transfer_active.

            STATE_WRITE: begin
                //to do. bus_write_req , transfer_active
            end

            STATE_WAIT_WRITE:
                //to do transfer_active.

            STATE_INC_ADDR:
                //to do transfer_active.

            STATE_DONE: begin
                //to do transfer_done , transfer_active.
            end

            default: transfer_active <= 1'b0;
        endcase
    end
end

```

```
    end
end
endmodule
```

LAB TASK:

TASK 1:

Objective:

Complete the implementation of a Direct Memory Access (DMA) Finite State Machine (FSM) controller by filling in the missing code sections. The DMA FSM is responsible for managing data transfers between memory-mapped peripherals and system memory without CPU intervention.

1. FSM State Implementation:

Complete the state machine logic for the following states:

- IDLE: Wait for transfer requests and configuration.
- READ: Initiate and control memory/peripheral read operations.
- WRITE: Execute write operations to destination.
- INCREMENT: Update address pointers and manage burst transfers.
- DONE: Signal transfer completion and prepare for next operation.

2. Missing Functionality to Implement:

Component	Required Implementation
State Transitions	Define all next-state logic based on current state and input signals
Control Signals	Generate appropriate mem_req, mem_write, peri_req signals
Burst Control	Implement burst counter management and burst transfer completion
Error Handling	Detect and handle memory/peripheral errors with proper recovery
Status Outputs	Drive dma_busy, dma_done, dma_error signals correctly
Internal Counters	Track transfer progress using byte_count and burst_counter
Handshake Protocol	Implement proper ack signal handling for memory/peripheral interfaces

TASK 2:

Objective:

Design a synchronous FIFO buffer that serves as the intermediary storage between the DMA's read and write operations.

FIFO Specifications

Depth: Configurable (typically 16 or 32 entries)

Data Width: Parameterizable (32-bit default)

Operation: Synchronous with clock and active-low reset

Count Output: Real-time tracking of stored data elements

FIFO Features to Implement

Write Interface:

Write enable signal

Write data input

Read Interface:

Read enable signal

Read data output

Status Monitoring:

Current element count

TASK 3:(protocol project part)

Objective:

Design and implement a complete DMA (Direct Memory Access) engine that transfers data between memory-mapped peripherals and system memory without CPU intervention, utilizing the provided memory module and DMA FSM (Finite State Machine) controller.

Task Requirements:

Integrate the provided memory module with the DMA controller to enable read/write operations during data transfers.

Utilize the given DMA FSM to manage the following states:

- IDLE: Wait for transfer request.
- READ: Read data from source address.
- WRITE: Write data to destination address.
- INCREMENT: Update address pointers and byte counter.
- DONE: Assert completion signal and return to IDLE.

Implement the complete DMA engine with the following features:

- Source and destination address registers.
- Transfer byte counter.
- Burst transfer capability (configurable burst size).
- Status registers (busy, done, error).
- Interrupt generation on transfer completion.

Support peripheral-to-memory transfer modes.

Ensure proper handshaking with the memory interface:

- Generate appropriate read/write enable signals.
- Handle address and data bus connections.

- Manage wait states or stall signals as needed.